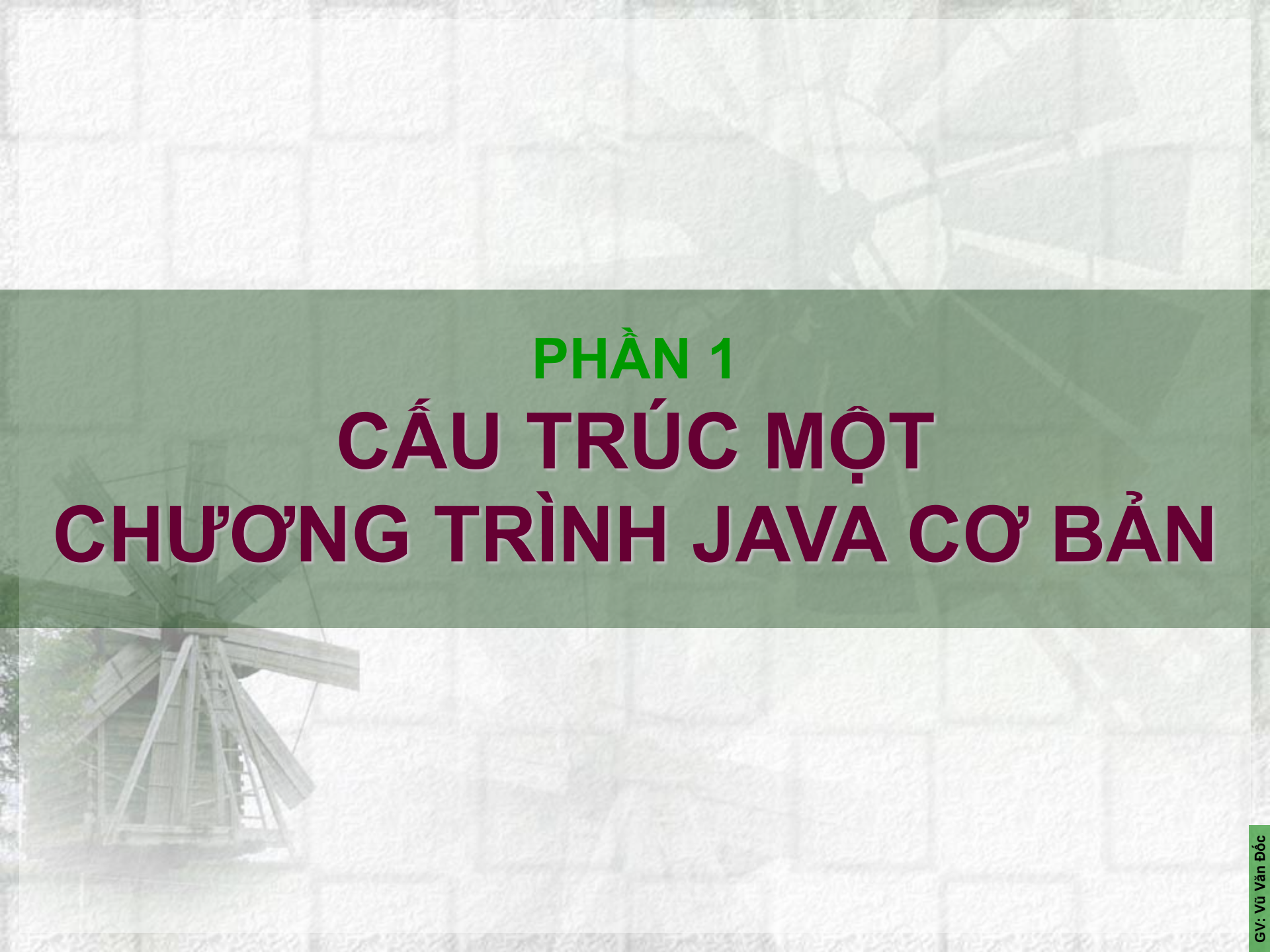


CÔNG NGHỆ JAVA
BÀI 2

JAVA CƠ BẢN



GIẢNG VIÊN:
Vũ Văn Đốc



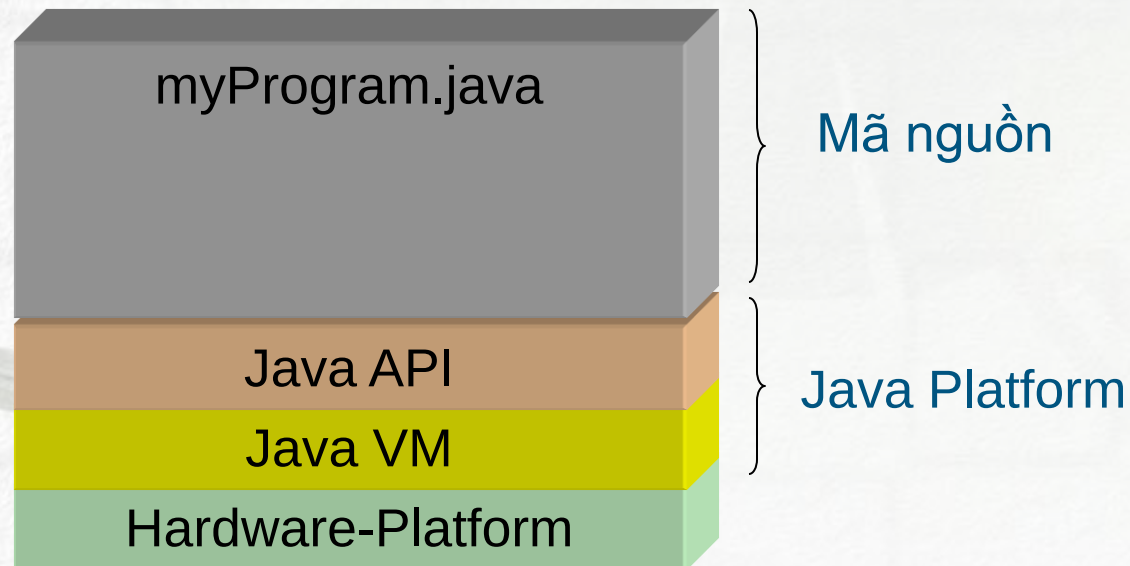
PHẦN 1

CẤU TRÚC MỘT

CHƯƠNG TRÌNH JAVA CƠ BẢN

KIẾN TRÚC CỦA JAVA

- Java Platform
 - Java Virtual Machine (Java VM)
 - Java Application Programming Interface (Java API)

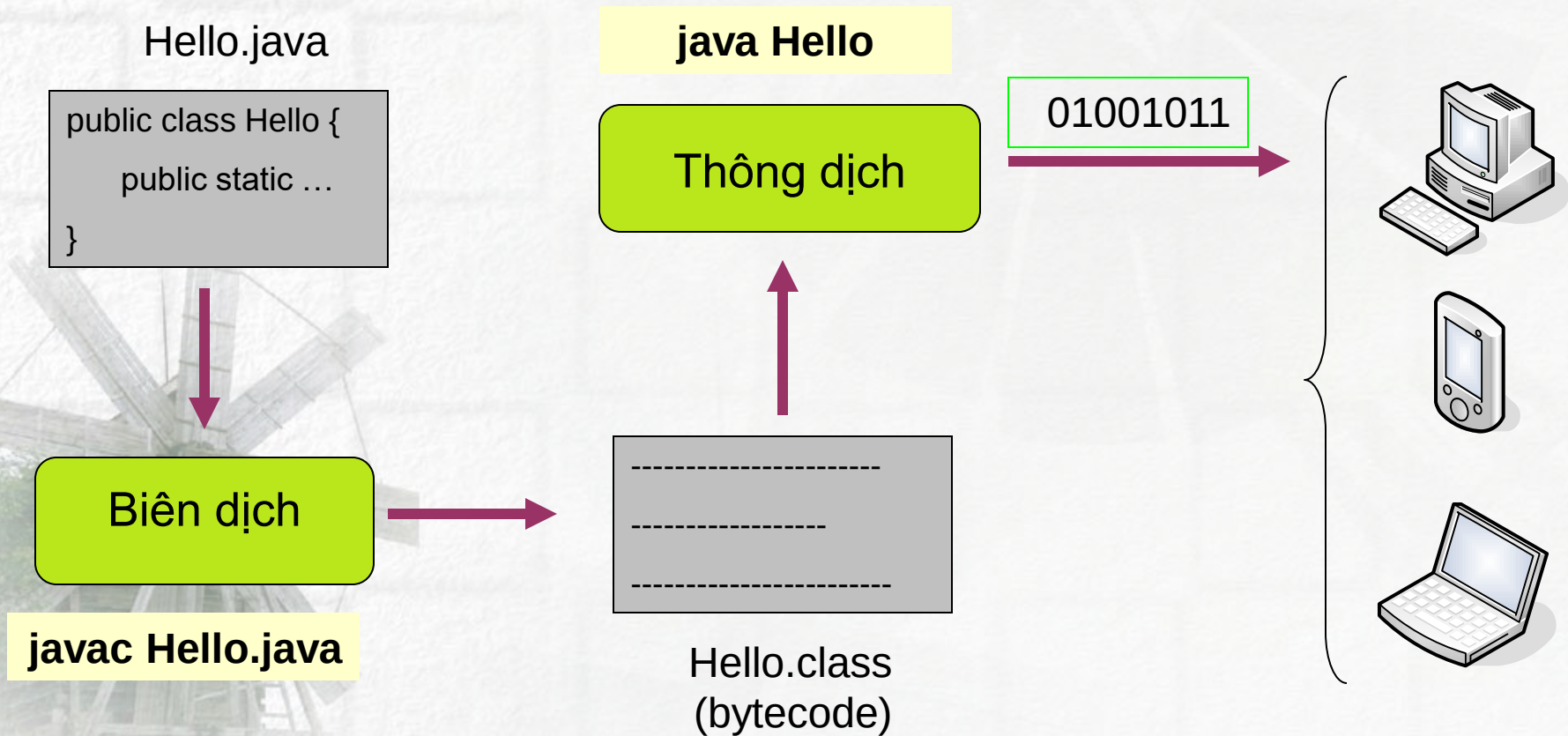


KIẾN TRÚC CỦA JAVA

- Thư viện lớp Java: bộ JDK bao gồm rất nhiều lớp chuẩn đã được xây dựng sẵn.
- Lập trình viên thường sử dụng các lớp chuẩn để phát triển ứng dụng.
- Các gói chuẩn của Java:
 - java.lang
 - java.applet
 - java.awt
 - java.io
 - java.util
 - java.net
 - java.awt.event
 - java.rmi
 - java.security
 - java.sql

CÁC BƯỚC PHÁT TRIỂN

- Các bước phát triển một chương trình bằng Java:



CẤU TRÚC MỘT CHƯƠNG TRÌNH CƠ BẢN

```
1 // Tên file : Hello.java
2 /* Tác giả : CNTT4A1*/
3
4 public class Hello
5 {
6 // Phương thức main, điểm bắt đầu của chương trình
7 public static void main( String args[ ] )
8 {
9     System.out.println( "Hello World" );
10
11 } // Kết thúc phương thức main
12
13 }
```

Tên lớp chứa hàm main phải giống tên file

Điểm bắt đầu và kết thúc của lớp

Dấu hiệu chú thích =>
Làm cho chương trình dễ hiểu hơn. Trình biên dịch sẽ bỏ qua những dòng có dấu chú thích

Khai báo lớp

Mỗi CT phải có ít nhất một khai báo lớp

Hiển thị dãy ký tự ra màn hình

Phương thức main() sẽ được gọi đầu tiên. Mỗi CT thực thi phải có một phương thức main()

Các câu lệnh phải kết thúc bằng dấu chấm phẩy

PHƯƠNG THỨC MAIN

- **Phương thức main():** là điểm bắt đầu thực thi một ứng dụng.
- Mỗi ứng dụng Java phải chứa một phương thức main có dạng như sau: **public static void main(String[] args)**
- Phương thức main chứa ba bổ từ đặc tả sau:
 - **public:** chỉ ra rằng phương thức main có thể được gọi bởi bất kỳ đối tượng nào.
 - **static:** chỉ ra rằng phương thức main là một phương thức lớp.
 - **void:** chỉ ra rằng phương thức main sẽ không trả về bất kỳ một giá trị nào.

CHÚ THÍCH TRONG JAVA

- Ngôn ngữ Java hỗ trợ ba kiểu chú thích sau:

`/* text */`

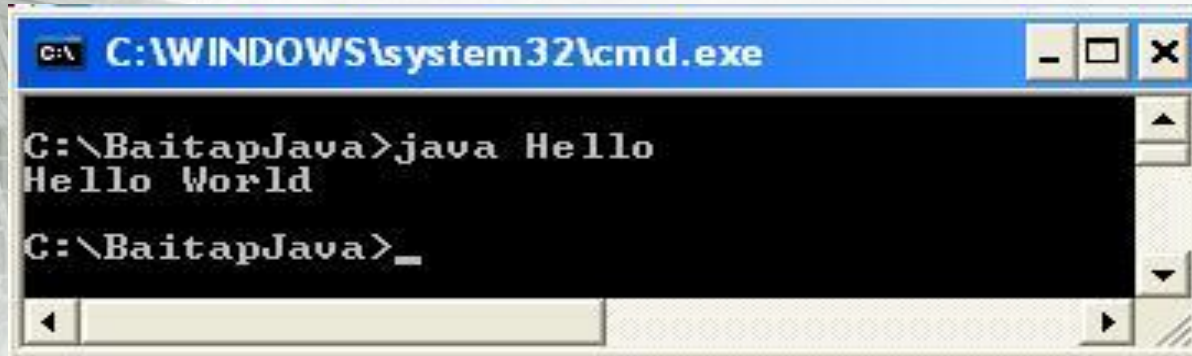
`// text`

`/** documentation */` công cụ javadoc trong bộ JDK sử dụng chú thích này để chuẩn bị cho việc tự động phát sinh tài liệu.

- Dấu mở và đóng ngoặc nhọn “{” và “}” là bắt đầu và kết thúc một khối lệnh.
- Dấu chấm phẩy “;” để kết thúc một dòng lệnh.
- Java được tổ chức theo lớp (class). Các lệnh và các hàm (kể cả hàm main) phải thuộc một lớp nào đó, chúng không được đứng bên ngoài của lớp.

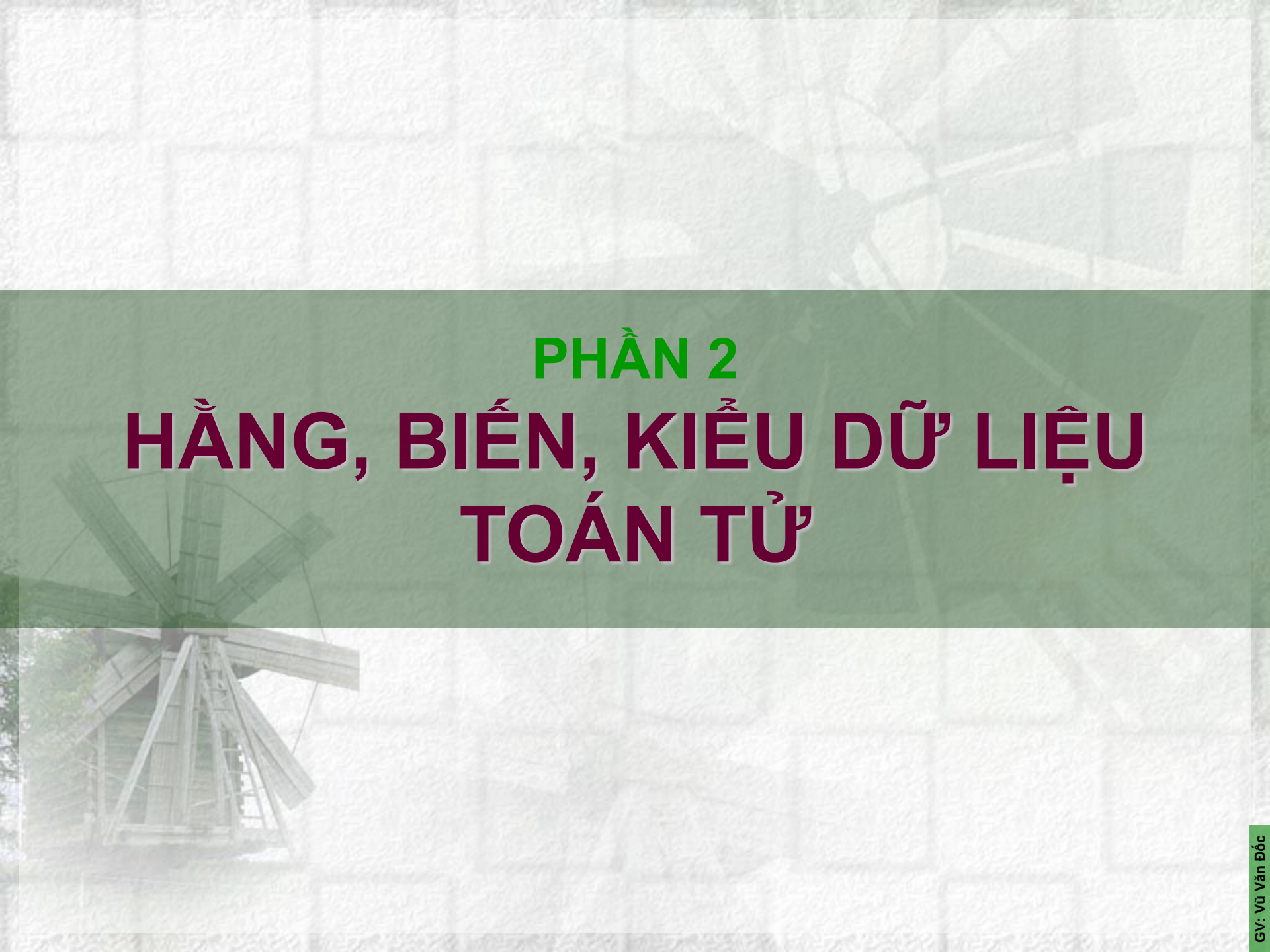
BIÊN DỊCH VÀ THỰC THI

- Biên dịch chương trình
 - Vào chế độ Console của Windows
 - Gõ câu lệnh ***javac Hello.java***
 - Nếu không có thông báo lỗi, file *Hello.class* sẽ được tạo ra
- Thực thi chương trình
 - Gõ câu lệnh ***java Hello*** (không cần *.class*)



A screenshot of a Windows Command Prompt window. The title bar shows the path `C:\WINDOWS\system32\cmd.exe`. The command prompt shows the following sequence of commands and output:

```
C:\BaitapJava>java Hello
Hello World
C:\BaitapJava>_
```



PHẦN 2

HẰNG, BIẾN, KIỂU DỮ LIỆU TOÁN TỬ

TỪ KHÓA (keyword)

- Từ khóa cho các kiểu dữ liệu cơ bản : **byte, short, int, long, float, double, char, boolean.**
- Từ khóa cho phát biểu lặp: **do, while, for, break, continue.**
- Từ khóa cho phát biểu rẽ nhánh: **if, else, switch, case, default, break.**
- Từ khóa đặc tả đặc tính một method: **private, public, protected, final, static, abstract, synchronized.**
- Hằng (literal): **true, false, null.**
- Từ khóa liên quan đến method: **return, void.**
- Từ khóa liên quan đến package: **package, import.**
- Từ khóa cho việc quản lý lỗi: **try, catch, finally, throw, throws.**
- Từ khóa liên quan đến đối tượng: **new, extends, implements, class, instanceof, this, super.**

TỪ KHÓA (keyword)

abstract	boolean	break	byte
case	catch	char	class
const	continue	default	do
double	else	extends	final
finally	float	for	goto
if	implements	import	instanceof
int	interface	long	native
new	package	private	protected
public	return	short	static
super	switch	synchronized	this
throw	throws	transient	try
void	volatile	while	

ĐỊNH DANH (identifier)

- Định danh là dùng biểu diễn tên của biến, của phương thức, của lớp.
- Trong Java, định danh có thể sử dụng ký tự chữ, ký tự số và ký tự dấu.
- Ký tự đầu tiên phải là ký tự chữ, dấu gạch dưới (_), hoặc dấu dollar (\$).
- Có sự phân biệt giữa ký tự chữ hoa và chữ thường.
Ví dụ: Hello, _prime, var8, tvLang

BIẾN (variable)

- Biến là vùng nhớ dùng để lưu trữ các giá trị của chương trình.
- Mỗi biến gắn liền với một kiểu dữ liệu và một định danh duy nhất gọi là tên biến.
- Tên biến thông thường là một chuỗi các ký tự (Unicode), ký số.
- Tên biến phải bắt đầu bằng một chữ cái, một dấu gạch dưới hay dấu dollar.
- Tên biến không được trùng với các từ khóa (xem lại các từ khóa trong java).
- Tên biến không có khoảng trắng ở giữa tên.
- Trong java, biến có thể được khai báo ở bất kỳ nơi đâu trong chương trình.

BIẾN (variable)

- **Cách khai báo**

<kiểu_dữ_liệu> <tên_biến>;

<kiểu_dữ_liệu> <tên_biến> = <giá_trị>;

- **Gán giá trị cho biến**

<tên_biến> = <giá_trị>;

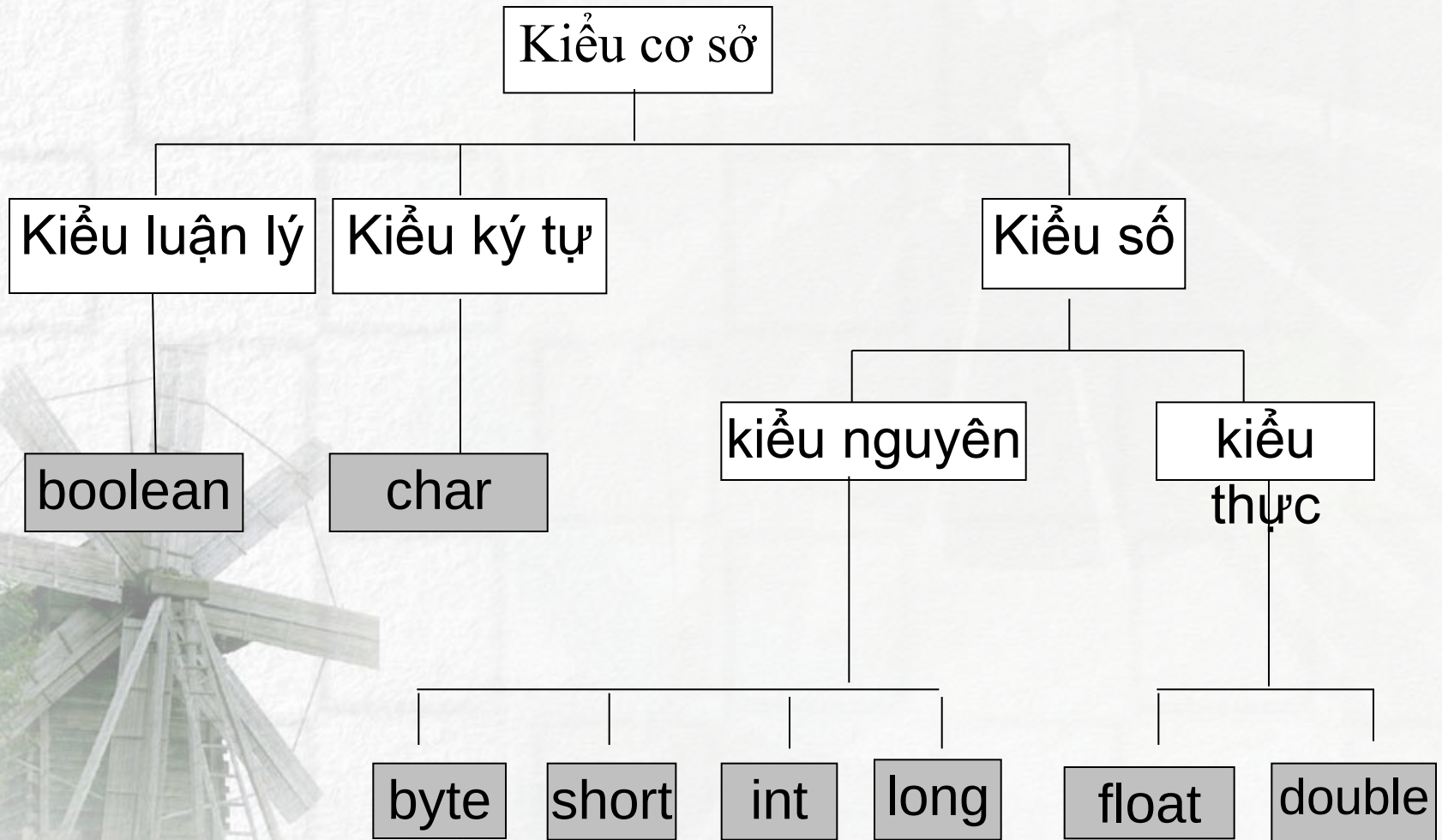
- **Biến công cộng (toàn cục):** là biến có thể truy xuất ở khắp nơi trong chương trình, thường được khai báo dùng từ khóa public, hoặc đặt chúng trong một class.
- **Biến cục bộ:** là biến chỉ có thể truy xuất trong khối lệnh nó khai báo

Kiểu dữ liệu (data type)

Kiểu dữ liệu:

- Kiểu dữ liệu cơ sở (Primitive data type)
- Kiểu dữ liệu tham chiếu hay dẫn xuất (reference data type)
 - Kiểu dữ liệu cơ sở của Java bao gồm các nhóm sau: số nguyên, số thực, ký tự, kiểu luận lý (logic)
 - Kiểu dữ liệu tham chiếu là các kiểu dữ liệu đối tượng. Ví dụ như: String, Byte, Character, Double, Boolean, Integer, Long, Short, Font,... và các lớp do người dùng định nghĩa.

Kiểu dữ liệu cơ sở (primitive type)



Kiểu dữ liệu cơ sở

- Kiểu số nguyên

Kiểu	Kích thước	Khoảng giá trị
byte	8 bits	-128...127
short	16 bits	-32768...32767
int	32 bits	$-2^{31} \dots 2^{31} - 1$
long	64 bits	$-2^{63} \dots 2^{63} - 1$

- Kiểu số thực

Kiểu	Kích thước	Khoảng giá trị
float	32 bits	-3.4e38...3.4e38
double	64 bits	-1.7e308...1.7e308

Kiểu dữ liệu cơ sở

- Kiểu **boolean**: Nhận giá trị true hoặc false
- Kiểu **char**: Kiểu ký tự theo chuẩn Unicode

Một số hằng ký tự:

Ký tự	Ý nghĩa
\b	Xóa lùi (BackSpace)
\t	Tab
\n	Xuống hàng
\r	Dấu enter
\”	Nháy kép
\’	Nháy đơn
\\	Số ngược
\f	Đẩy trang
\uxxxx	Ký tự unicode

Kiểu dữ liệu cơ sở

Kiểu số nguyên

- Bốn kiểu số nguyên khác nhau là: byte, short, int, long
- Kiểu mặc định của các số nguyên là kiểu int
- Không có kiểu số nguyên không dấu
- Không thể chuyển biến kiểu int và kiểu boolean như trong ngôn ngữ C/C++

VD: int x = 0;
 long y=100;
 int a=1,b,c;

Kiểu dữ liệu cơ sở

- Kiểu số nguyên:

```
boolean b = false;
```

```
if (b == 0)
```

```
{
```

```
    System.out.println("Xin chao");
```

```
}
```

Lúc biên dịch, đoạn chương trình trên sẽ báo lỗi vì ta không được so sánh biến kiểu boolean với biến kiểu int.

Kiểu dữ liệu cơ sở

Một số lưu ý đối với các phép toán trên số nguyên:

- Nếu hai toán hạng kiểu long thì kết quả là kiểu long.
- Một trong hai toán hạng không phải kiểu long sẽ được chuyển thành kiểu long trước khi thực hiện phép toán.
- Nếu hai toán hạng đầu không phải kiểu long thì phép tính sẽ thực hiện với kiểu int.
- Các toán hạng kiểu byte hay short sẽ được chuyển sang kiểu int trước khi thực hiện phép toán.

Kiểu dữ liệu cơ sở

Kiểu dấu chấm động:

- Kiểu float có kích thước 4 byte và giá trị mặc định là 0.0f
- Kiểu double có kích thước 8 byte và giá trị mặc định là 0.0d
- Khai báo và khởi tạo giá trị cho các biến kiểu dấu chấm động:

float x = 100.0/7;

double y = 1.56E6;

Kiểu dữ liệu cơ sở

Một số lưu ý với các phép toán trên số dấu chấm động:

- Nếu mỗi toán hạng đều có kiểu dấu chấm động thì phép toán chuyển thành phép toán dấu chấm động.
- Nếu có một toán hạng là double thì các toán hạng còn lại sẽ được chuyển thành kiểu double trước khi thực hiện phép toán.
- Biến kiểu float và double có thể ép chuyển sang kiểu dữ liệu khác (trừ kiểu boolean).

Kiểu dữ liệu cơ sở

- Kiểu ký tự
 - Biểu diễn các ký tự trong bộ mã Unicode
 - $2^{16} = 65536$ ký tự khác nhau :
 - từ '\u0000' đến '\uFFFF'
- Kiểu luận lý (boolean)
 - Hai giá trị: **true** hoặc **false**
 - Giá trị mặc định: **false**
 - Không thể chuyển thành kiểu nguyên và ngược lại

Kiểu dữ liệu cơ sở

- Kiểu mảng

- Khai báo: `int[] iarray; hoặc int iarray[];`
- Cấp phát: `iarray = new int[100];`
- Khởi tạo:

`int[] iarray = {1, 2, 3, 5, 6};`

`char[] carray = {'a', 'b', 'c'};`

Chú ý: Luôn khởi tạo hoặc cấp phát mảng trước khi sử dụng

- Một số khai báo không hợp lệ:

`int[5] iarray;`

`int iarray[5];`

Kiểu dữ liệu cơ sở

Kiểu mảng

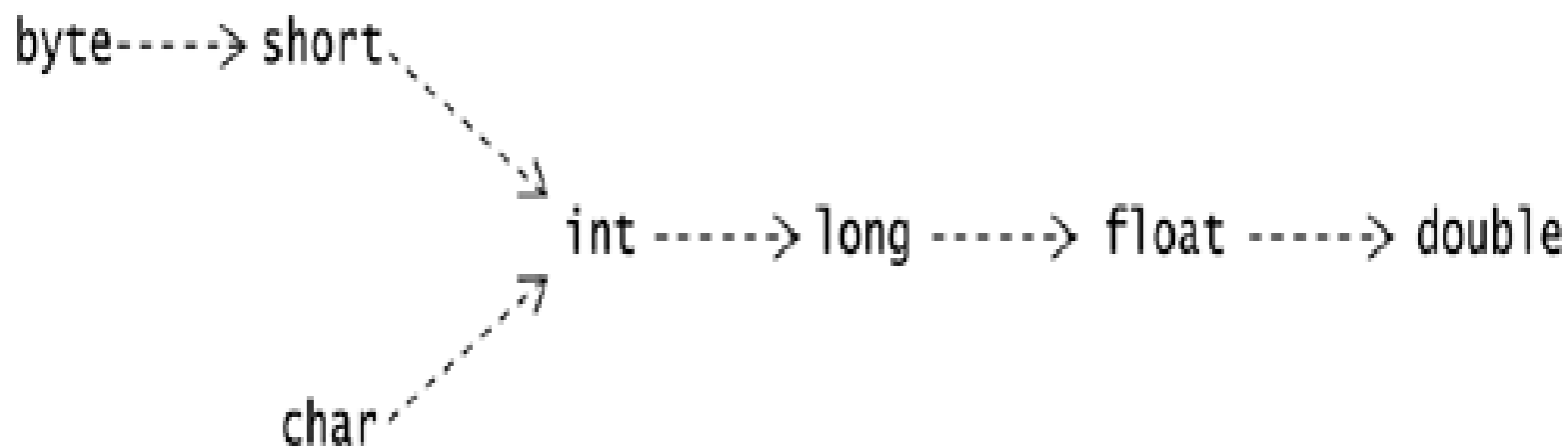
- Truy cập mảng
 - `iarray[3] = 0;`
 - `carray[1] = 'z';`

Chú ý: Chỉ số của mảng được tính từ 0

- Lấy số phần tử mảng:
`iarray.length`

Kiểu dữ liệu cơ sở

- Khi gặp phải sự không tương thích kiểu dữ liệu chúng ta phải tiến hành chuyển đổi kiểu dữ liệu cho biến hoặc biểu thức



Kiểu dữ liệu cơ sở

- **Toán tử ép kiểu:**

`<tên biến> = (kiểu_dữ_liệu) <tên_biến>;`

`float fNum = 2.2;`

`int iCount = (int) fNum`

- **Ép kiểu rộng** (widening conversion): từ kiểu nhỏ sang kiểu lớn (không mất mát thông tin)
- **Ép kiểu hẹp** (narrow conversion): từ kiểu lớn sang kiểu nhỏ (có khả năng mất mát thông tin)

HẲNG (LITERAL)

- Hằng là một giá trị bất biến trong chương trình
- Tên hằng được đặt theo qui ước giống như tên biến
- Tiếp vĩ ngữ: *l, L, f, F, d, D*

int i=1;

long i=1L;

- Hằng ký tự: là một ký tự đơn nằm giữa 2 dấu nháy đơn.

HẰNG (LITERAL)

- **Hàng chuỗi:** là tập hợp các ký tự được đặt giữa hai dấu nháy kép “”. Một hàng chuỗi không có ký tự nào là một hàng chuỗi rỗng.

Ví dụ: “Hello World”

Lưu ý: Hàng chuỗi không phải là một kiểu dữ liệu cơ sở nhưng vẫn được khai báo và sử dụng trong các chương trình

TOÁN TỬ (OPERATOR)

- Toán tử số học

Toán tử	Ý nghĩa
+	Cộng
-	Trừ
*	Nhân
/	Chia nguyên
%	Chia dư
++	Tăng 1
--	Giảm 1

TOÁN TỬ (OPERATOR)

- Toán tử quan hệ & logic

Toán tử	Ý nghĩa
==	So sánh bằng
!=	So sánh khác
>	So sánh lớn hơn
<	So sánh nhỏ hơn
>=	So sánh lớn hơn hay bằng
<=	So sánh nhỏ hơn hay bằng
	OR (biểu thức logic)
&&	AND (biểu thức logic)
!	NOT (biểu thức logic)

TOÁN TỬ (OPERATOR)

- Toán tử gán (assignment)

Toán tử	Ví dụ	Giải thích
=	int c=3, d=5, c=4;	
+=	c += 7	c=c+7
-=	d -= 4	d=d-4
=	e=5	e=e*5
/=	f/=3	f=f/3
%=	g%=9	g=g%9

TOÁN TỬ (OPERATOR)

- Toán tử điều kiện

<điều kiện> ? <biểu thức 1> : <biểu thức 2>

int x = 10;

int y = 20;

int Z = (x < y) ? 30 : 40;

Ví dụ tìm giá trị lớn nhất của 3 số a, b, c:

Max = (a > b ? a : b) > c ? (a > b ? a : b) : c

ĐỘ ƯU TIÊN CỦA CÁC PHÉP TOÁN

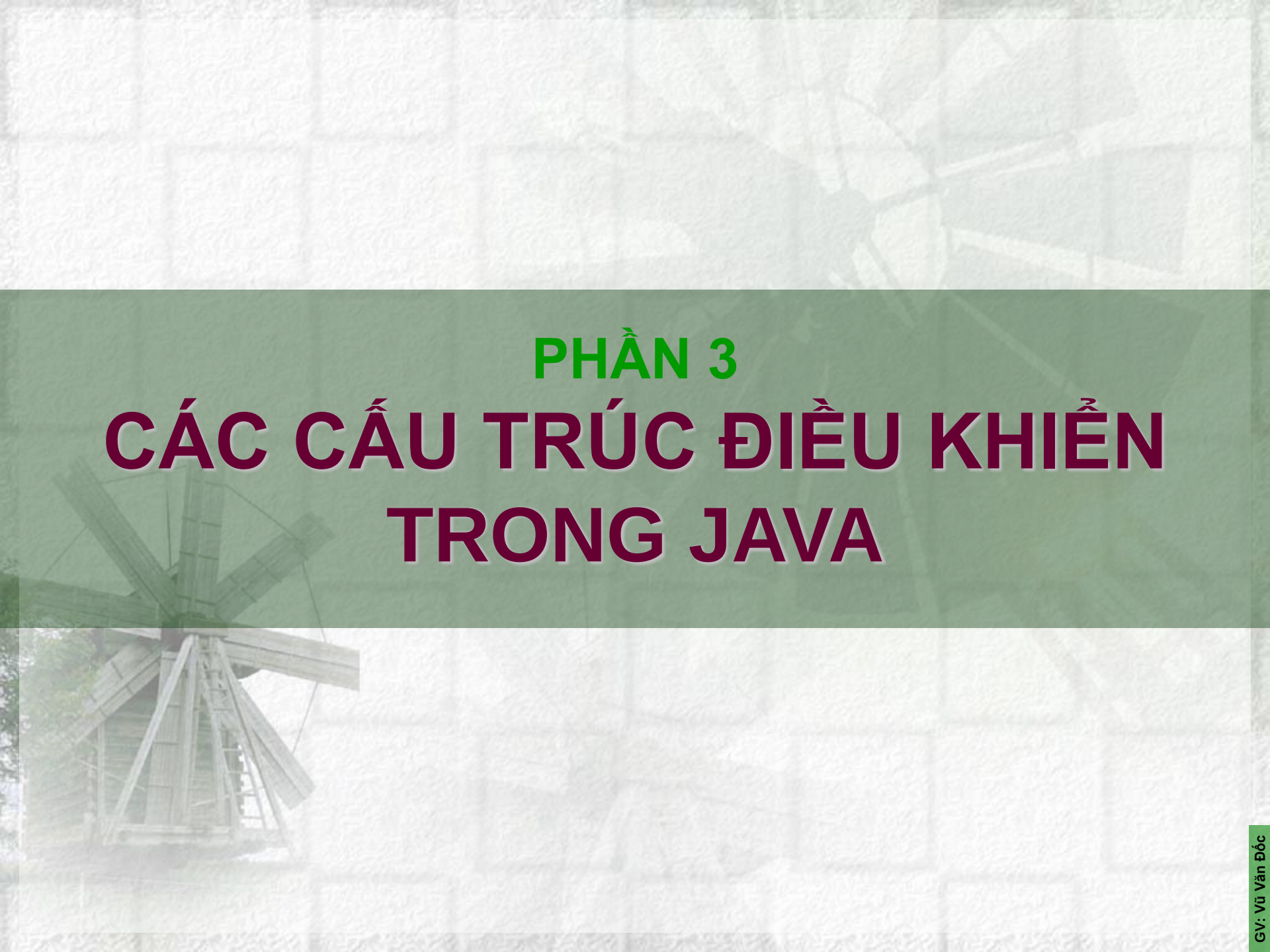
- Độ ưu tiên của các phép toán trong ngôn ngữ Java cũng gần giống như ngôn ngữ C/C++. Thứ tự ưu tiên từ trái qua phải và từ trên xuống dưới như bảng sau:

1	.	[]	()	
2	++	--	!	~
3	*	/	%	
4	+	-		
5	<<	>>		
6	<	>	<=	>=
7	==	!=		
8	&			
9	^			
10	&&			
11				
12	?:			
13	=			

MỘT VÍ DỤ VỀ PHÉP TOÁN

Ví dụ:

```
import java.lang.*;
import java.io.*;
class VariableDemo
{
    static int x, y;
    public static void main(String[] args)
    {
        x = 10;
        y = 20;
        int z = x+y;
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("z = x + y =" + z);
        System.out.println("So nho hon la so:" + Math.min(x, y));
        char c = 80;
        System.out.println("ky tu c la: " + c);
    }
}
```

PHẦN 3

CÁC CẤU TRÚC ĐIỀU KHIỂN

TRONG JAVA

CÁC CẤU TRÚC ĐIỀU KHIỂN

- Điều khiển rẽ nhánh:
 - Mệnh đề **if-else**
 - Mệnh đề **switch-case**
- Vòng lặp (Loops):
 - Vòng lặp **while**
 - Vòng lặp **do-while**
 - Vòng lặp **for**

MỆNH ĐỀ IF - ELSE

Cú pháp:

if (điều kiện)

{ câu lệnh; }

Hoặc *if* (điều kiện)

{ câu lệnh 1; }

else

{ câu lệnh 2; }

điều kiện: Biểu thức Boolean như toán tử so sánh. Biểu thức này trả về giá trị True hoặc False

Câu lệnh 1: Các dòng lệnh được thực thi khi giá trị trả về là True

Câu lệnh 2: Các câu lệnh được thực thi nếu điều kiện trả về giá trị False

MỆNH ĐỀ IF - ELSE

- Ví dụ:

```
public class TestIf
{
    public static void main( String args[ ] )
    {
        int num =10;
        if (num %2 == 0)
            System.out.println (num+ "is an even number");
        else
            System.out.println (num + "is an odd number");
        }
    }
```

MỆNH ĐỀ SWITCH - CASE

Cú pháp

switch ()

{*biểu thức*

case 'giá trị 1': Câu lệnh 1; break;

case 'giá trị 2': Câu lệnh 2; break;

:

case 'giá trị N': Câu lệnh N; break;

default: Câu lệnh N+1;

}

biểu thức – Biểu thức chứa một giá trị xác định

giá trị 1, giá trị 2 giá trị N: Các giá trị hằng số phù hợp với giá trị trên biến **biểu thức**.

default: Từ khóa tùy chọn được sử dụng để chỉ rõ các câu lệnh nào được thực hiện chỉ khi tất cả các trường hợp nhận giá trị False

MỆNH ĐỀ SWITCH - CASE

- Ví dụ về mệnh đề switch/case

```
class SwitchDemo
{
    public static void main(String args[])
    {
        int day =4;
        switch(day)
        {
            case 0 : System.out.println("Sunday"); break;
            case 1 : System.out.println("Monday"); break;
            case 2 : System.out.println("Tuesday"); break;
            case 3 : System.out.println("Wednesday"); break;
            case 4 : System.out.println("Thursday"); break;
            case 5 : System.out.println("Friday"); break;
            case 6 : System.out.println("Saturday"); break;
            default: System.out.println("Invalid day of week");
        }
    }
}
```


VÒNG LẶP WHILE

- Cú pháp:

while (<biểu thức boolean>)
 { <khối lệnh>; }

Kiểm tra <biểu thức boolean>, true thì thực hiện khối lệnh, false thì thoát khỏi vòng lặp, chuyển sang câu lệnh tiếp.

```
class WhileDemo
{
    public static void main(String args[])
    {
        Int a = 5, fact = 1;
        while (a >= 1)
        {
            fact *= a;
            a--;
        }
        System.out.println("The Factorial of 5 is " + fact);
    }
}
```

VÒNG LẶP DO - WHILE

- Cú pháp:

do

{

 <khối lệnh>;

} **while** <biểu thức boolean>;

Thực hiện <khối lệnh> sau đó xuống kiểm tra <biểu thức boolean>, **true** thì quay lên thực hiện khối lệnh, **false** thì thoát khỏi vòng lặp chuyển sang câu lệnh tiếp.

```
// Tính tổng các số lẻ từ 1 đến 100
```

```
int tong = 0, i=1;
```

```
do
```

```
{
```

```
    tong+=i; i+=2;
```

```
} while (i<=100);
```

```
System.out.println(tong);
```

VÒNG LẶP FOR

- Vòng lặp for

- for(<khởi tạo>; <điều kiện lặp>; <bước nhảy>)
{ <khối lệnh>; }

// Chương trình tính tổng các số lẻ từ 1 đến 100

```
public class TestFor
{
    public static void main(String[] args)
    {
        int tong = 0;
        for(int i=1; i<=100; i+=2)
            tong+=i;
        System.out.println(tong);
    }
}
```


VÒNG LẶP FOR

- Vòng lặp for
 - for(<tên biến>: <tên mảng>)
{ <khối lệnh>; }

// Chương trình in ra các phần tử của mảng nguyên a:

```
public class TestFor
{
    public static void main(String[] args)
    {
        for (int temp:a)
        {
            System.out.print(temp + " ");
        }
    }
}
```

LỆNH break, continue

- Lệnh break sẽ chấm dứt quá trình lặp mà không thực hiện nốt phần còn lại của cấu trúc lặp
- Lệnh continue sẽ dừng không thực thi phần còn lại của thân vòng lặp mà chuyển điều khiển về điểm bắt đầu của vòng lặp, để thực hiện lần lặp tiếp theo.

// BreakTest.java

```
public class BreakTest
```

```
{
```

```
    public static void main( String args[] )
```

```
    { int count; // control variable also used after loop terminates
```

```
      for ( count = 1; count <= 10; count++ ) // loop 10 times
```

```
        { if ( count == 5 ) // if count is 5,
```

```
          break; // terminate loop
```

```
      System.out.printf( "%d ", count ); } // end for
```

```
      System.out.printf( "\nBroke out of loop at count = %d\n", count );
```

```
    } // end main } // end class BreakTest
```


LỆNH break, continue

```
// BreakTest.java
public class BreakTest
{
    public static void main( String args[] )
    {
        int count;
        for ( count = 1; count <= 10; count++ ) // loop 10 times
        {
            if ( count == 5 ) // if count is 5,
                break; // terminate loop
            System.out.printf( "%d ", count ); // end for
            System.out.printf( "\nBroke out of loop at count = %d\n",
                count );
        } // end main // end class BreakTest
    }
}
```


LỆNH break, continue

```
// ContinueTest.java
public class ContinueTest
{
    public static void main( String args[] )
    {
        int count; // control variable also used after loop terminates
        for ( count = 1; count <= 10; count++ ) // loop 10 times
            { if ( count == 5 ) // if count is 5,
                continue; // terminate loop
            }
        System.out.printf( "%d ", count ); // end for
        System.out.println( "\nUsed continue to skip printing 5" );
    } // end main
} // end class ContinueTest
```

Thảo luận

1. Trong cấu trúc lệnh if-else đơn (1 if và 1 else) thì có ít nhất một khối lệnh (của if hoặc của else) được thực hiện. Đúng hay sai?
2. Trong cấu trúc lệnh switch-case, khi không dùng “default” thì có ít nhất một khối lệnh được thực hiện. Đúng hay sai?
3. Trong cấu trúc lệnh switch-case, khi dùng “default” thì có ít nhất một khối lệnh được thực hiện. Đúng hay sai?
4. Trong cấu trúc lệnh while, khối lệnh được thực hiện ít nhất một lần ngay cả khi điều kiện có giá trị False. Đúng hay sai?
5. Trong cấu trúc lệnh do-while, khối lệnh được thực hiện ít nhất một lần ngay cả khi điều kiện có giá trị False. Đúng hay sai?
6. Trong cấu trúc lệnh for, khối lệnh được thực hiện ít nhất một lần ngay cả khi điều kiện có giá trị False. Đúng hay sai?

Thảo luận

7. Cho biết kết quả thu được khi thực hiện đoạn chương trình sau?

```
class me{  
    public static void main(String args[]){  
        int sales = 82;  
        int profit = 20;  
        System.out.println((sale +profit)/(10*5));  
    }  
}
```

8. Cho biết đoạn chương trình sau thực hiện vòng lặp bao nhiêu lần và kết quả in ra là gì?

```
class me{  
    public static void main(String args[]){  
        int i = 0; int sum = 0; do{  
            sum += i;  
            i++;  
        } while(i <= 10);  
        System.out.println(sum);  
    }  
}
```


Thảo luận

9. Cho biết đoạn chương trình sau thực hiện vòng lặp bao nhiêu lần và kết quả in ra là gì?

```
class me{  
    public static void main(String args[ ]){  
        int i = 5; int sum = 0;  
        do {  
            sum += i;  
            i++;  
        } while(i < 5);  
        System.out.println(sum);  
    }  
}
```

Thảo luận

10. Cho biết hai đoạn chương trình sau in ra kết quả giống hay khác nhau?

```
class me1{  
    public static void main(String args[]){  
        //int i = 0;  
        int sum = 0;  
        for(int i=0; i<5; i++){  
            sum += i;  
        } System.out.println(sum);  
        System.out.println(i);  
    } }
```

và:

```
class me2{  
    public static void main(String args[]){  
        int i = 0;  
        int sum = 0;  
        for( ; i<5; ++i){  
            sum += i;  
        } System.out.println(sum);  
        System.out.println(i);  
    } }
```

Thảo luận

11. Tính tổng dãy số: $1 + 1/2 - 1/3 + 1/4 - \dots + 1/100$

12. Cho số nguyên n , kiểm tra xem n có là số nguyên tố không? In ra tất cả các số nguyên tố có trong mảng số nguyên a cho trước?



NHẬP DỮ LIỆU TỪ BÀN PHÍM

Sử dụng lớp BufferedReader

- ✓ BufferedReader là một lớp dùng để đọc dữ liệu từ bàn phím hay từ file.
- ✓ Có thể dùng lớp này để đọc một **chuỗi** một **mảng** hoặc một **kí tự**.
- ✓ Tham số đầu vào của BufferedReader có thể là InputStreamReader hoặc FileReader (để đọc từ file)
- ✓ **Một số phương thức của lớp BufferedReader:**
 - **read()** : đọc một kí tự.
 - **readLine()** : đọc một dòng text.
- ✓ Chuyển đổi kiểu dữ liệu từ text sang int dùng phương thức lớp Integer, Float, Double... và phương thức parseInt(), parseFloat, parseDouble...
- ✓ Sử dụng thư viện **java.io.***

NHẬP DỮ LIỆU TỪ BÀN PHÍM

Sử dụng lớp `BufferedReader`

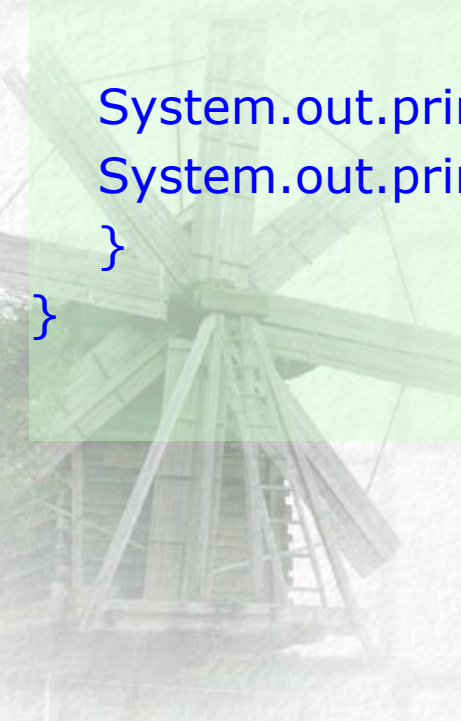
- Ví dụ nhập một số nguyên và một số thực

```
import java.io.*;
public class TestInput
{
    public static void main(String[] args) throws Exception
    {
        BufferedReader inStream =
            new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Nhap mot so nguyen:");
        String siNumber = inStream.readLine();
        int iNumber = Integer.parseInt(siNumber);
```


NHẬP DỮ LIỆU TỪ BÀN PHÍM

Nhập dữ liệu từ bàn phím

```
System.out.print("Nhap mot so thuc:");  
String sfNumber = inStream.readLine();  
float fNumber = Float.parseFloat(sfNumber);  
//float fNumber = Float.parseFloat(inStream.readLine());  
  
System.out.println("So nguyen:" + iNumber);  
System.out.println("So thuc:" + fNumber);  
}  
}
```



NHẬP DỮ LIỆU TỪ BÀN PHÍM

Sử dụng lớp Scanner

Người ta có thể sử dụng lớp Scanner để nhập các số int, float, double, boolean, byte...như sau:

// Tạo một đối tượng Scanner

```
Scanner nhapso = new Scanner(System.in);
```

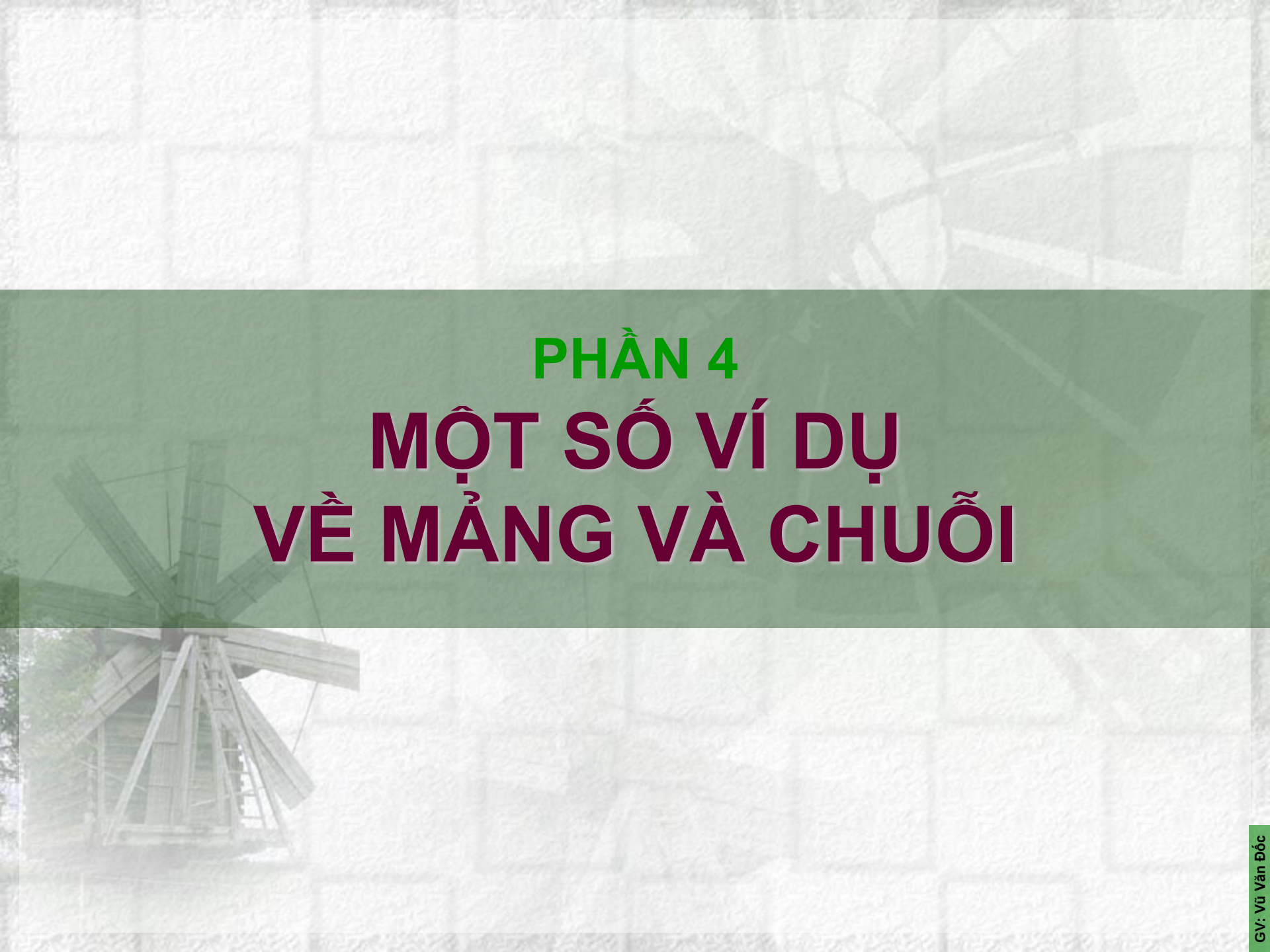
//Nhập một số nguyên

```
iNumber = nhapso.nextInt();
```

Một số phương thức:

- nextInt()
- nextFloat()
- nextBoolean()
- nextByte()
- nextLine()

Các phương thức này nằm trong lớp: `java.util.*`;



PHẦN 4

MỘT SỐ VÍ DỤ VỀ MẢNG VÀ CHUỖI

MẢNG

Mảng là tập hợp nhiều phần tử có cùng tên, cùng kiểu dữ liệu và mỗi phần tử trong mảng được truy xuất thông qua chỉ số của nó trong mảng.

Khai báo mảng:

```
<kiểu dữ liệu> <tên mảng>[];  
<kiểu dữ liệu>[] <tên mảng>;
```

Ví dụ:

```
int arrInt[];  
int[] arrInt;  
int[] arrInt1, arrInt2, arrInt3;
```


Khởi tạo mảng:

Chúng ta có thể khởi tạo giá trị ban đầu cho các phần tử của mảng khi nó được khai báo.

Ví dụ:

```
int arrInt[] = {1, 2, 3};
```

```
char arrChar[] = {'a', 'b', 'c'};
```

```
String arrString[] = {"ABC", "EFG", "GHI"};
```

Truy cập mảng:

- Chỉ số mảng trong Java bắt đầu từ 0. Vì vậy phần tử đầu tiên có chỉ số là 0, và phần tử thứ n có chỉ số là $n-1$. Các phần tử của mảng được truy xuất thông qua chỉ số của nó được đặt giữa cặp dấu ngoặc vuông [].

- Ví dụ:**

```
int arrInt[] = {1, 2, 3};
```

```
int x = arrInt[0]; // x sẽ có giá trị là 1.
```

```
int y = arrInt[1]; // y sẽ có giá trị là 2.
```

```
int z = arrInt[2]; // z sẽ có giá trị là 3.
```

MẢNG

- Nhập và xuất giá trị các phần tử của một mảng các số nguyên:

```
class ArrayDemo
{
    public static void main(String args[])
    {
        int arrInt[] = new int[10];
        int i;
        for(i = 0; i < 10; i = i+1)
            arrInt[i] = i;
        for(i = 0; i < 10; i = i+1)
            System.out.println("This is arrInt[" + i + "]: " + arrInt[i]);
    }
}
```


MẢNG

- Tìm phần tử có giá trị nhỏ nhất (Min) và lớn nhất (Max) trong một mảng.

```
class MinMax2
```

```
{    public static void main(String args[])
    {    int nums[] = { 99, -10, 100123, 18, -978, 5623, 463, -9, 287, 49 };
        int min, max;
        min = max = nums[0];
        for(int i=1; i < 10; i++)
        {
            if(nums[i] < min) min = nums[i];
            if(nums[i] > max) max = nums[i];
        }
        System.out.println("Min and max: " + min + " " + max);
    }
}
```

MẢNG

- Nhập và xuất giá trị của các phần tử trong một mảng hai chiều.

```
class TwoD_Arr
{
    public static void main(String args[])
    {
        int t, i;
        int table[][] = new int[3][4];
        for(t=0; t < 3; ++t)
        {
            for(i=0; i < 4; ++i)
            {
                table[t][i] = (t*4)+i+1;
                System.out.print(table[t][i] + " ");
            }
            System.out.println();
        }
    }
}
```

CHUỖI TRONG JAVA (STRING)

- Trong những ngôn ngữ lập trình khác (C chẳng hạn), một chuỗi được xem như một mảng các ký tự.
- Trong java thì khác, java cung cấp một lớp String để làm việc với đối tượng dữ liệu chuỗi cùng các thao tác trên đối tượng dữ liệu này.



CHUỖI

- Nhập ký tự từ bàn phím:

```
import java.io.*;
class InputChar
{
    public static void main(String args[])
    {
        char ch = '';
        try{
            ch = (char) System.in.read();
        }
        catch(Exception e){
            System.out.println("Nhập lỗi!");
        }
        System.out.println("Ky tu vua nhap:" + ch);
    }
}
```

CHUỖI

- **Nhập dữ liệu số**

```
import java.io.*;
class inputNum
{
    public static void main(String[] args)
    {
        int n=0;
        try { BufferedReader in = new BufferedReader(
            new InputStreamReader(System.in));

            String s;
            s = in.readLine();
            n = Integer.parseInt(s);
        }
        catch(Exception e){
            System.out.println("Nhập dữ liệu bị lỗi !");
        }
        System.out.println("Bạn vừa nhập số:" + n);
    }
}
```

CHUỖI

- Tạo đối tượng chuỗi

```
class StringDemo
{
    public static void main(String args[])
    {
        // Tao chuoi bang nhieu cach khac nhau
        String str1 = new String("Chuoi trong java la nhung Objects.");
        String str2 = "Chung duoc xay dung bang nhieu cach khac nhau.";
        String str3 = new String(str2);
        System.out.println(str1);
        System.out.println(str2);
        System.out.println(str3);
    }
}
```


CHUỖI

- **Minh họa một số thao tác cơ bản trên chuỗi**

// Chương trình minh họa các thao tác trên chuỗi ký tự

class StrOps

```
{    public static void main(String args[])
    {    String str1 = "Java là chọn lựa số một cho lập trình ứng dụng Web.";
        String str2 = new String(str1);
        String str3 = "Java hỗ trợ đối tượng String để xử lý chuỗi";
        int result, idx;
        char ch;
        System.out.println("str1:" + str1);
        System.out.println("str2:" + str2);
        System.out.println("str3:" + str3);
        System.out.println("Chiều dài của chuỗi str1 là:" + str1.length());
        // Hiển thị chuỗi str1, mỗi lần một ký tự.
        System.out.println();
        for(int i=0; i < str1.length(); i++)
            System.out.print(str1.charAt(i));
```

CHUỖI

```
System.out.println();
if(str1.equals(str2))
    System.out.println("str1 == str2");
else
    System.out.println("str1 != str2");
if(str1.equals(str3))
    System.out.println("str1 == str3");
else
    System.out.println("str1 != str3");
result = str1.compareTo(str3);
if(result == 0)
    System.out.println("str1 = str3 ");
else
if(result < 0)
    System.out.println("str1 < str3");
else
    System.out.println("str1 > str3");
```


CHUỖI

```
// Tao chuoai moi cho str4
String str4 = "Mot Hai Ba Mot";
idx = str4.indexOf("Mot");
System.out.println("str4:" + str4);
System.out.println(
    "Vi tri xuat hien dau tien cua chuoai con 'Mot' trong str4: " + idx);
idx = str4.lastIndexOf("Mot");
System.out.println("Vi tri xuat hien sau cung cua
chuoai con 'Mot' trong str4:" + idx);
}
```


CHUỖI

- Chương trình nhập vào một chuỗi và in ra chuỗi nghịch đảo của chuỗi nhập.

```
import java.lang.String;
import java.io.*;
public class InverstString
{
    public static void main(String arg[])
    {
        System.out.println("\n *** CHUONG TRINH IN CHUOI
                                NGUOC *** ");

        try
        {
            System.out.println("\n *** Nhap chuoi:");
            BufferedReader in = new BufferedReader(new
                InputStreamReader(System.in));

            // Class BufferedReader cho phép đọc text từ luồng nhập ký
            //tự, tạo bộ đệm cho những ký tự để hỗ trợ cho việc đọc
            //những ký tự, những mảng hay những dòng.
```

CHUỖI

```
// Doc 1 dong tu BufferReadered ket thuc bang dau ket thuc dong.  
String str = in.readLine();  
System.out.println("\n Chuoi vua nhap la:" + str);  
// Xuat chuoai nghich dao  
System.out.println("\n Chuoi nghich dao la:");  
for (int i=str.length()-1; i>=0; i--)  
{  
    System.out.print(str.charAt(i));  
}  
}  
catch (IOException e)  
{  
    System.out.println(e.toString());  
}  
}  
}
```

CHUỖI

- Lấy chuỗi con của một chuỗi

```
class SubStr
{
    public static void main(String args[])
    {
        String orgstr = "Mot Hai Ba Bon";
        // Lay chuoai con dung ham
        // public String substring(int beginIndex, int
        // endIndex)
        String substr = orgstr.substring(4, 7);
        System.out.println("Chuoai goc: " + orgstr);
        System.out.println("Chuoai con: " + substr);
    }
}
```


CHUỖI

- **Mảng các chuỗi**

```
class StringArray
{
    public static void main(String args[])
    {
        String str[] = {"Mot", "Hai", "Ba", "Bon" };
        System.out.print("Mang goc: ");
        for(int i=0; i < str.length; i++)
            System.out.print(str[i] + " ");
        System.out.println("\n");
        // Thay doi chuoi
        str[0] = "Bon"; str[1] = "Ba";
        str[2] = "Hai"; str[3] = "Mot";
        System.out.print("Mang thay doi:");
        for(int i=0; i < str.length; i++)
            System.out.print(str[i] + " ");
        System.out.print("\n");
    }
}
```

HẾT

CHƯƠNG 2

